

Image Mosaic

Badr Kassou, Diljit Chokar, Dorine Lenoir

Bruface Programme (Master of Science in Electrical Engineering)
Université libre de Bruxelles (ULB) & Vrije Universiteit Brussel (VUB)
Brussels, Belgium

Abstract—This project presents a MATLAB implementation of an image mosaic generation algorithm. The objective is to reconstruct a target image using a dataset of smaller images, called tiles. The target image is divided into regular blocks, and each block is replaced by the tile that best matches its visual characteristics. Several matching methods were implemented and compared, including average RGB color, luminance, texture analysis, RGB histogram distances, and a combined descriptor. To improve the visual coherence of the final mosaic, color correction and blending were added as optional post-processing steps. A graphical user interface was also developed to allow the user to select the target image, load a tile dataset, adjust the tile size, choose the matching method, and control correction parameters. The generated mosaics were evaluated visually and quantitatively using PSNR, SSIM, and computation time. The results show how feature extraction, similarity measurement, and color adjustment can be combined to produce recognizable and visually coherent image mosaics.

Index Terms—image mosaic, tile matching, color correction, blending, MATLAB, image processing

I. INTRODUCTION

Image mosaics are based on a visual effect in which many small images are arranged so that, when viewed from a distance, they form a larger picture. This creates two levels of interpretation: the local level, where each tile can still be recognized individually, and the global level, where the overall arrangement reproduces the target image. This makes image mosaics interesting from both an artistic and an image processing perspective. From a technical point of view, the main challenge is to select, for each region of the target image, a tile that is visually consistent with that region. This requires extracting relevant features from both the target image and the tile dataset, then comparing them using an appropriate similarity measure. Classical image mosaic methods generally rely on a regular grid and improve the final result using color correction techniques. More advanced approaches, such as Jigsaw Image Mosaics, extend the concept by using arbitrarily shaped tiles and by considering gaps, overlaps, and deformations.

The aim of this project is to study a grid-based mosaic generation approach and to evaluate how different visual descriptors influence the quality of the final result.

II. DESCRIPTION OF OUR ALGORITHM

This section presents the main steps of the proposed image mosaic algorithm, from image preprocessing to tile matching and final post-processing.

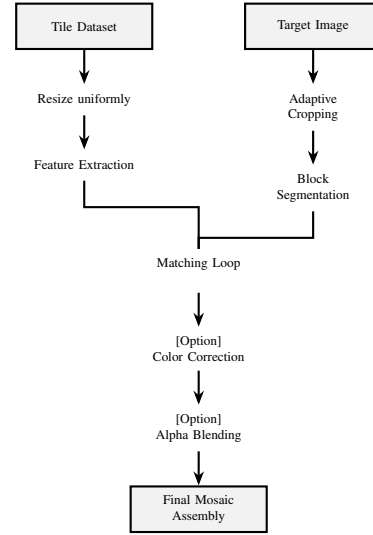


Fig. 1. Architecture and operational pipeline of the block-based image mosaic generator.

A. Preprocessing and Spatial Normalization Phase

Before any feature extraction can occur, data harmonization is required to standardize dimensions and prevent matrix mismatch errors. On the one hand, the `loadTileDataset` function imports tiles from a local directory (supporting .png, .jpg, or raw .mat batches), converts them to double, and duplicates the channels of any grayscale images to enforce a uniform RGB format. These tiles are then uniformly resized by `resizeTiles` to the user-selected dimensions. However, to eliminate edge artifacts and aspect distortions in the final mosaic, the target image undergoes adaptive cropping. This process truncates its original dimensions to the maximum dimensions that are perfectly divisible by the tile size.

B. Feature Engineering Models

In order to generate the mosaic, several feature extraction methods were implemented to describe the visual content of the target blocks and database tiles.

1) *Baseline Color Descriptor (`mean_rgb`)*: This method is the baseline technique of our application. It simplifies each block of the target image and each tile by calculating the average color of their pixels. For a block, the average is calculated for the three channels (red, green and blue). Next, the `matchTile` function compares the target block vector with the vectors of all tiles. It chooses the closest tile by looking for the smallest Euclidean distance. This method is

very fast to execute, but it completely ignores shapes, outlines, and textures inside blocks.

2) *Luminance and Local Contrast Descriptor (luminance)*: This method is used to capture the brightness and contrast of the blocks, independently of their colors. First, the block is converted to grayscale to find its luminance Y using the standard formula:

$$Y(i, j) = 0.2989 \cdot R(i, j) + 0.5870 \cdot G(i, j) + 0.1140 \cdot B(i, j)$$

The final feature vector has 5 dimensions : $f = [\bar{R}, \bar{G}, \bar{B}, \bar{Y}, \sigma_Y]$. The strength of this method relies on σ_Y , the standard deviation of the luminance. If this value is close to 0, the block is uniform and flat. If it's high, the block contains many variations and is textured. This prevents the algorithm from confusing a smooth block with a textured block that shares the exact same average color.

3) *Sobel-Based Texture Descriptor (texture)*: This method captures sharp details, edges and contours inside the blocks. The algorithm applies the Sobel filter to the luminance channel to detect where the brightness changes rapidly, which indicates an edge. Next, the normalized gradient magnitude is calculated for each pixel to measure the edge strength. The final feature vector stores the average RGB colors, alongside the mean and standard deviation of these gradient. This allow the application to easily separate blocks with sharp outlines from smooth background.

4) *Histogram Descriptor (histogram_bhattacharyya)*: Instead of using simple averages, this method analyses the full distribution of colors. For each block and tile, the algorithm builds a normalized histogram with 8 bins for the RGB channels, combining them into a single feature vector. While multiple distance metrics were initially tested during development, the Bhattacharyya distance consistently provided the best visual results. Consequently, the final version of `matchTile_histogram` uses this metric exclusively, using vectorized operations to select the tile that minimizes the distance across the three concatenated channels.

5) *Combined Feature Descriptor (combined)*: This method combines colors, brightness, and texture into a single feature vector to achieve the highest possible matching accuracy. The algorithm merges the vectors from the previous methods : the 3 average RGB colors, the 3-channel color histograms, the average luminance, the luminance standard deviation, and the mean and standard deviation of the Sobel gradient.

C. Optical Post-Processing Algorithms

To improve the final visual quality of the mosaic and smooth the transitions between tiles, we implemented 2 post-processing techniques.

1) *Blending Strength (applyBlending)*: This technique balances the visual appearance between the selected tile and the original target block. The parameter α linearly combines both images. At $\alpha=0$, the pure tile is displayed at 100%, keeping details sharp but potentially creating visible block

borders. At $\alpha=1$, the mosaic effect disappears as the original block is fully restored. It keeps 80% of the tile's details while injecting 20% of the original block's colors. This blends the color differences and visually softens the boundaries between tiles, making the global mosaic look much smoother from a distance.

2) *Color Correction (applyColorCorrection)*: This technique improves global color coherence by adjusting the tile's brightness to match the target block. Both images are converted to the YIQ color space to isolate the luminance channel. Following a shift-and-scale rule, the tile's average luminance is shifted to match the target's average luminance. If this shift causes underflow, a scale factor based on the minimum luminance is applied instead. The corrected Y channel is then combined with the original I and Q channels and converted back to RGB to smooth out color mismatches.

III. EXPERIMENTAL RESULTS

A. Experimental Setup and Input Data

The experiments were carried out using a single target image Fig. 2 and a dataset composed of flower images. The flower dataset was selected because it contains a large variety of colors, which is important for producing visually coherent mosaics. Before generating the mosaic, all tile images were converted to RGB format, normalized, and resized to a fixed size of 30×30 pixels. This ensured that the comparison between the different matching strategies remained fair and consistent. All tile images were resized to a fixed size before the mosaic generation. The tested methods include the baseline average RGB descriptor, the luminance descriptor, the Sobel-based texture descriptor, the histogram-based descriptor using the Bhattacharyya distance, and the combined descriptor. For each method, the mosaic was generated using the same post-processing parameters. Color correction was enabled to improve global color coherence, and blending was applied with a moderate value of α in order to soften transitions between neighbouring tiles while keeping the tile images recognizable. The performance of each method was evaluated visually and quantitatively using PSNR, SSIM, and computation time.



Fig. 2. Target image used for the mosaic generation.

B. Visual Comparison of Matching Methods

Fig. 3 shows the target image and the mosaics generated with the five selected matching methods. The target image contains a strong contrast between the dark sky and the brighter rock formations, which makes it a useful test case for evaluating color, luminance, and structural preservation. The mean RGB method provides a recognizable approximation of the target image, but it gives the lowest visual and numerical quality because it only considers the average color of each block. The luminance and texture descriptors improve the reconstruction by adding information about brightness variations and local edges. The histogram method based on the Bhattacharyya distance produces a more coherent result because it compares the full color distribution instead of only relying on average values. The combined descriptor also produces a balanced mosaic by using color, luminance and texture features together. In this implementation, the combined descriptor uses weights $[3, 1, 1]$, giving more importance to the color component while still including luminance and texture information.

C. Quantitative Evaluation

The different matching methods were also evaluated quantitatively using three criteria: PSNR, SSIM, and computation time. PSNR measures the pixel-wise reconstruction error between the generated mosaic and the cropped target image. It is useful for estimating how closely the mosaic reproduces the target image in terms of color intensity. However, PSNR does not fully reflect the perceived visual quality of the result. For this reason, SSIM was also used as a complementary metric, since it evaluates structural similarity by considering local contrast, luminance, and image organization. Finally, computation time was measured to compare the practical efficiency of the descriptors and to assess their suitability for interactive use in the graphical interface. Fig. 4 shows that the mean RGB method obtains the lowest PSNR and is also the slowest method in this experiment. The luminance, texture, histogram Bhattacharyya and combined descriptors all provide a clear improvement in reconstruction quality. The luminance descriptor gives the highest PSNR, while the histogram Bhattacharyya method obtains a very similar PSNR with a slightly higher computation time.

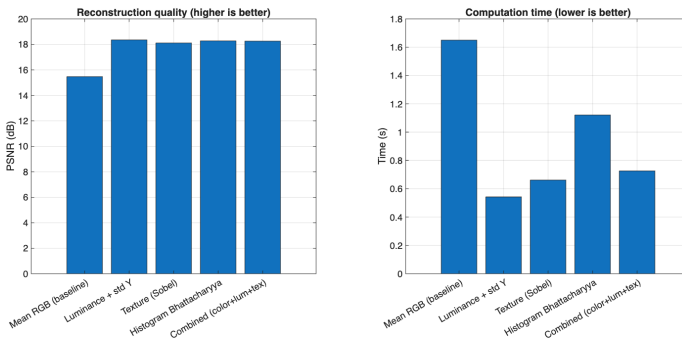


Fig. 4. Quantitative comparison of the implemented methods using PSNR and computation time.

Fig. 5 shows that the histogram Bhattacharyya method achieves the highest SSIM value. This indicates that, although luminance performs best in terms of pixel-wise error, the histogram-based descriptor better preserves the global structure of the target image. The combined descriptor gives a balanced result, but it does not outperform the best individual methods in this specific experiment.

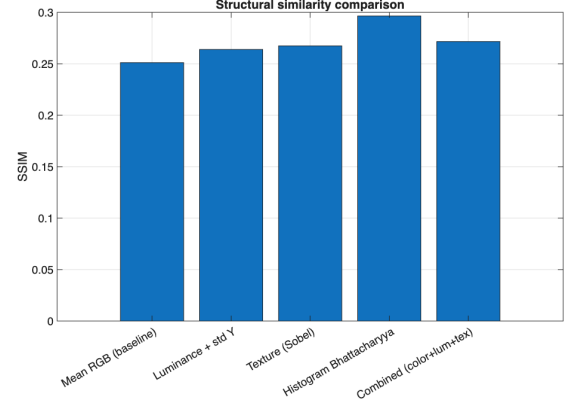


Fig. 5. Structural similarity comparison of the implemented matching methods using SSIM.

D. Graphical User Interface

To easily manipulate the matching experiments, test different parameters in real time, and instantly visualize the results, we developed a dedicated Graphical User Interface (GUI) using MATLAB App Designer. This application integrates a step-by-step textual tutorial and a functional color-coded system to seamlessly guide the user through each stage of the process, from image importing to the final mosaic generation.

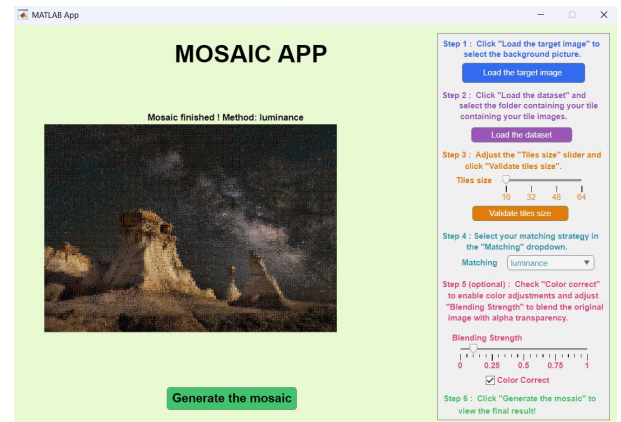


Fig. 6. Final Graphical User Interface (GUI) of the MOSAIC APP.

To run the application, ensure the main directory is active and simply type `mosaic_app` in the MATLAB Command Window.

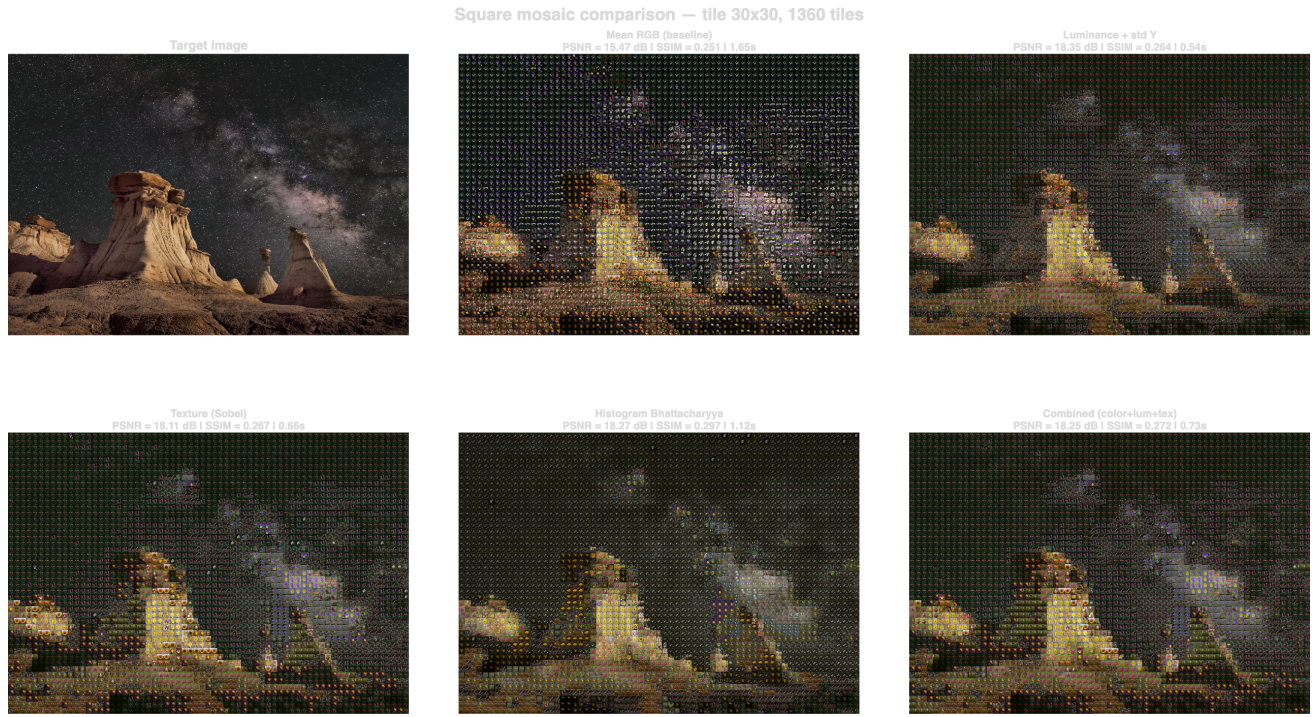


Fig. 3. Visual comparison of the target image and the generated mosaics using the selected matching methods: mean RGB, luminance, texture, histogram Bhattacharyya, and the combined descriptor.

IV. CONCLUSION

This project implemented a MATLAB image mosaic generator based on block-wise tile matching. Several descriptors were compared, including mean RGB, luminance, texture, histogram Bhattacharyya, and a combined method. Color correction and blending were also added to improve the visual coherence of the final mosaic.

The results show that more advanced descriptors generally produce better mosaics than simple average color matching. In the tested case, luminance achieved the highest PSNR, while the Bhattacharyya histogram method obtained the highest SSIM. A graphical user interface was also successfully implemented, allowing the user to select the target image, load a tile dataset, adjust the tile size, choose the matching method, and control the post-processing parameters interactively.

Future work could include better weight optimization, reduced tile repetition, larger datasets, and true irregular or jigsaw-shaped tiles.

REFERENCES

- [1] J. Kim and F. Pellacini, “Jigsaw Image Mosaics,” *ACM Transactions on Graphics (TOG) - Proceedings of ACM SIGGRAPH 2002*, vol. 21, no. 3, pp. 657–664, July 2002[cite: 1].
- [2] A. Finkelstein and M. Range, “Image Mosaics,” in *Electronic Publishing, Artistic Imaging and Digital Typography*, R. D. Hersch, J. André, and H. Brown, Eds. Heidelberg: Springer-Verlag, 1998, pp. 11–22[cite: 2, 365, 368].
- [3] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Toronto, ON, Canada, Tech. Rep., 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>

- [4] M.-E. Nilsback and A. Zisserman, “Automated flower classification over a large number of classes,” in *Proc. IEEE Indian Conf. Computer Vision, Graphics and Image Processing*, 2008.
- [5] W. Suggs, “Bisti Badlands under the Milky Way,” Ah-Shi-Sle-Pah Wilderness, New Mexico, Landscape Photograph, 2026.

AUTHOR CONTRIBUTIONS

The contributions of the authors to this project are distributed as follows:

Badr Kassou: Responsible for the preprocessing phase, including the spatial normalization, adaptive cropping, and data loading functionalities.

Diljit Chokar: Implemented and evaluated the core tile matching algorithms and similarity metrics, including the feature engineering models.

Dorine Lenoir: Designed and developed the Graphical User Interface (GUI) using MATLAB App Designer, integrated the system components, and adjusted the execution parameters to ensure overall system coherence.